第十讲 - 代码组织 (编程练习)

张建章

阿里巴巴商学院

杭州师范大学

2026-09



1 业务问题引入

2 从单文件到模块

3 模块的定义与导入

4 导入规则与常见错误

5 使用Python标准库

6 第三方库与运行环境

7 综合业务实践

8 总结与课后拓展

Python 程序设计基础

第10讲 代码组织 速记卡

让程序更清晰、更好找、更容易复用！





1 为什么要学代码组织？

举个例子：做“订单处理”时，如果把文件读取、数据清洗、金额计算、汇总统计、报表保存等所有代码都写在一个 main.py 里，会出现：



```
main.py
# 读取文件 ...
# 清洗数据 ...
# 计算金额 ...
# 汇总统计 ...
# 保存报表 ...
```

→ 代码很长
→ 不好修改
→ 不易复用

关键结论：代码组织（把不同功能拆分到合适的文件中），能让代码更容易找到、更好复用、更方便测试和维护。

2 从单文件到模块

把不同的功能分到不同的 .py 文件中，形成清晰的项目结构：

```
order_project/
├── main.py
├── order_utils.py
└── report_utils.py
```

各文件的主要职责：

- **main.py**：组织流程（按顺序调用各功能）
- **order_utils.py**：清洗订单、校验规则、计算实付金额
- **report_utils.py**：按渠道汇总、生成报表

一句话：一个文件，承担一个主要职责。

3 模块的定义与导入

- **模块**（module）就是一个 .py 文件，形来放一组相关的函数、变量等。

```
# order_utils.py
def calculate_payable(amount,
    discount_rate):
    return amount * (1 - discount_rate)
```

- **常见的导入方式：**

```
import order_utils
from order_utils import calculate_payable
import order_utils as ou
```

✅ **推荐写法：**先 import 模块名，再用 模块名.函数名 调用（如：order_utils.calculate_payable(...)）。

❌ 不要使用 from ... import *（容易导致名字冲突，也不利于代码的可读性）。

4 导入规则与常见错误

- **Python 查找模块的常见顺序：**
  1. **内置模块：**Python 自带的（如 math、csv）。
  2. **当前项目目录 / Notebook 目录：**会在你当前的文件夹中找 .py 文件。
  3. **当前 Python 环境中的库：**你安装的第三方库（如 pandas 等）。
- **常见问题与检查方法：**
  - **ModuleNotFoundError：**检查文件名、模块名是否正确，以及当前的工作目录是否是项目目录。
  - **同名冲突：**不要把文件命名为 csv.py / json.py / random.py（会遮盖 Python 自带的模块）。
  - **Notebook 修改模块后没变化：**可以重新加载模块：

```
import importlib
importlib.reload(order_utils)
```

5 使用Python标准库

- **标准库：**Python 自带的一组实用工具，安装 Python 后就可以直接使用，通常不需要单独安装。
- **常用示例：**
  - **pathlib：**处理文件路径（如拼接路径、判断文件是否存在）
  - **csv：**读取订单表（如 .csv 格式的订单数据）
  - **statistics：**进行基本统计（如计算平均值）

```
from pathlib import Path
import csv
from statistics import mean
```

小提示：当现有的工具已经能满足需求时，优先使用标准库，简单、稳定、可靠！

6 第三方库与运行环境

- **第三方库：**由社区开发的扩展工具，提供更多强大的功能，例如：

pandas（数据分析）	requests（发送网络请求）	openpyxl（读写 Excel）

- **一般需要先安装，常见安装方式：**

```
%pip install pandas
python -m pip install pandas
```

- **运行环境**（简单理解）：
  - 每个项目最好有独立的环境，避免不同项目所需文库互相影响；
  - .venv：用来保存本项目的 Python 解释器和已安装的库；
  - requirements.txt：用来记录项目依赖（需要哪些库）。



注意：如果安装成功，但仍然 import 失败，很可能是“安装的环境”和“运行的环境”不是同一个。

7 综合业务实践：订单渠道汇总项目

读取 orders.csv → 计算 实付金额 → 跳过 无效订单 → 按渠道 汇总

- **示例订单数据（简化）：**

订单号	渠道	金额(元)	折扣	有效
01001	天猫	1180.00	0.20	✓
01002	京东	360.00	0.10	✓
01003	抖音	200.00	0.00	✗

- **汇总结果（有效订单）：**
  - 天猫：944.00元
  - 京东：324.00元
- **模块协作：**
  - **main.py：**连接整个流程（读取 → 计算 → 过滤 → 汇总）；
  - **order_utils.py：**计算实付金额、校验订单是否有效；
  - **report_utils.py：**按渠道汇总，得到各渠道的销售总额。
- **简化代码示例（main.py）：**

```
import order_utils
import report_utils
orders = order_utils.load_orders('orders.csv')
valid_orders = order_utils.process_orders(orders)
summary = report_utils.summarize_by_channel(valid_orders)
print(summary)
```

8 学习提醒 / 易错点

- 模块名来自文件名（如 order_utils.py 的模块名是 order_utils）。
- 模块名尽量用小写和下划线（如 order_utils）。
- 先查标准库，再考虑第三方库。
- 修改模块后，Notebook 可能要重新加载（如使用 importlib.reload(...)）。
- 别把所有代码都堆在 main.py。
- 用 requirements.txt 记录依赖，方便自己和他人复现环境。
- 主程序写流程，细节放到模块里，让代码更清晰、可复用。



加油！
从今天开始，养成良好的代码组织习惯，写出更专业、更高效的 Python 程序！

记住：代码组织不是背语法，而是把“职责拆分 + 导入连接 + 环境管理”做好，让程序更清晰、更稳定、更容易协作。

一个文件承担了全部工作

业务任务

运营人员每天读取订单文件，清洗无效数据，计算实付金额，按销售渠道汇总结果，并将报表保存到文件中。

```python
# main.py
def read_orders(path): ...
def clean_orders(orders): ...
def calculate_payables(orders): ...
def summarize_by_channel(orders): ...
def save_report(summary, path): ...

orders = read_orders("data/orders.txt")
orders = clean_orders(orders)
orders = calculate_payables(orders)
summary = summarize_by_channel(orders)
save_report(summary, "output/report.txt")
```

当前的代码组织方式：程序可以正常运行，但文件读取、数据清洗、业务计算和报表输出的代码都放在 `main.py` 中。业务规则增加后，这个文件会继续变长。

需求增长后的维护问题

业务变化	单文件程序面临的问题
增加平台佣金规则	需要在较长的文件中寻找并修改相关计算代码。
退款分析也需要清洗订单	容易复制已有代码，随后出现多份不一致的清洗规则。
修改报表的字段和格式	需要浏览与报表无关的文件读取和业务计算代码。
让合作者协助修改	需要提供较大的代码上下文，修改结果也更难检查。

本讲需要解决的问题

怎样在不改变业务逻辑的前提下，按职责拆分代码，再通过导入把不同文件连接成一个完整程序。

代码组织的目标

让每个文件负责一类清晰的任务，使代码更容易查找、复用、测试和修改。

订单项目的文件结构

拆分依据

把共同完成一类任务的函数放在同一个文件中。主程序负责组织执行流程，其他文件负责具体业务功能。

```
order_project/
  main.py
  order_utils.py
  report_utils.py
```

文件	主要职责
`main.py`	组织处理流程，调用其他文件提供的函数。
`order_utils.py`	清洗订单数据，校验业务规则并计算实付金额。
`report_utils.py`	汇总订单数据，并将统计结果写入报表。

拆分后的变化：业务逻辑没有改变，但每个文件的职责更加清晰。修改报表时，通常只需要查看 `report_utils.py` 。

模块、包和库

概念	常见形式	作用
模块	一个 `.py` 文件	保存相关的函数、变量和可执行语句。例如，`order_utils.py` 是一个模块。
包	组织多个模块的目录	用于管理规模更大的代码。包目录通常包含 `__init__.py` 。
库	一组可复用的代码	这是一个较宽泛的称呼。一个库可以包含模块和包。可能由Python或第三方开发者提供。

三个概念之间的关系

模块和包是Python组织及导入代码时使用的概念。库是对可复用代码集合的通用称呼。

本讲的学习重点

先掌握模块：把函数保存到独立的 `.py` 文件中，再通过导入在主程序中使用这些函数。

定义订单工具模块

在项目文件夹中新建 `order_utils.py`：

```python
"""订单数据校验和计算工具。"""

def calculate_payable(amount, discount_rate):
    if amount < 0:
        raise ValueError("订单金额不能为负数")
    if not 0 <= discount_rate <= 1:
        raise ValueError("折扣率必须在0和1之间")
    return amount * (1 - discount_rate)
```

什么是模块

包含Python定义和语句的 `.py` 文件就是一个模块。模块名来自文件名，因此该文件对应的模块名是 `order_utils` 。

模块命名

模块名通常使用简短的小写字母，必要时使用下划线。避免使用 `csv.py` 、 `json.py` 等标准库模块名称。

导入模块并调用函数

```python
# main.py
import order_utils

payable = order_utils.calculate_payable(640.0, 0.15)

print(f"实付金额：{payable:.2f}元") # 实付金额：544.00元
```

导入方式	调用方式与特点
`import order_utils`	使用 `order_utils.calculate_payable()`，函数来源清楚，优先推荐。
`from order_utils import ...`	直接使用 `calculate_payable()`，适合只需要少量明确函数的情况。
`import order_utils as ou`	使用 `ou.calculate_payable()`，适合名称较长或已有通用别名的模块。

不建议导入全部名称： `from order_utils import *` 会把多个名称直接放入当前程序，难以判断函数来源，也可能覆盖已有名称。

模块被导入时会发生什么

在 `order_utils.py` 末尾加入测试代码：

```
if __name__ == "__main__":
    result = calculate_payable(640.0, 0.15)
    print(f"测试结果：{result:.2f}元")
```

运行方式	`__name__` 的值	测试代码
`python order_utils.py`	`"__main__"`	执行
在 `main.py` 中导入	`"order_utils"`	不执行

这种写法的作用

函数可以被其他程序安全导入。直接运行模块时，又可以执行简单测试或演示代码。

导入时仍会执行的内容

Python会执行模块最外层的语句。因此，可复用模块的最外层通常只放函数定义、常量和必要的初始化代码。

Python在哪里寻找模块

下面的两个文件位于同一项目文件夹中：

```
order_project/
  main.py
  order_utils.py
```

在 `main.py` 中可以直接导入：

```
import order_utils
```

查找位置	说明
Python内置模块	首先检查Python自身提供的内置模块。
程序所在位置	运行 `.py` 文件时，Python会查找脚本所在的目录；在Notebook中，Python会查找ipynb文件所在的目录。
Python环境中的目录	继续查找标准库和当前Python环境中已经安装的第三方库。

导入问题的检查方法 I

```python
from pathlib import Path
import order_utils

print(Path.cwd())
print(order_utils.__file__)
```

现象	主要检查内容
`ModuleNotFoundError`	检查模块名、文件名和工作目录，确认模块位于Python的查找范围中。
导入后缺少预期函数	查看模块的 `__file__`，确认Python导入了预期文件。检查当前目录是否存在同名文件。
修改模块后结果未变化	Notebook通常保留已经导入的模块。重新启动内核，或者重新加载该模块。

导入问题的检查方法 II

Notebook中重新加载模块

```
import importlib
importlib.reload(order_utils)
```

重新加载后，再次调用模块中的函数。

避免模块名称冲突

不要把自己的文件命名为 `random.py`、`json.py` 或 `csv.py`，否则可能遮蔽同名的标准库模块。

标准库提供的可复用工具

什么是Python标准库

Python标准库随Python一起提供，包含大量可直接导入的模块。使用标准库模块通常不需要另外安装软件包。

模块	订单处理任务	本例使用的功能
`pathlib`	构造订单文件路径	使用 `Path` 表示文件和目录。
`csv`	读取结构化订单数据	使用 `DictReader` 按字段名读取每行数据。
`statistics`	计算订单统计指标	使用 `mean()` 计算平均订单金额。

选择工具的基本顺序

先确认Python标准库是否已经提供所需功能。标准库无法满足任务时，再考虑编写自定义模块或安装第三方库。

标准库也需要导入

`len()` 和 `print()` 等内置函数可以直接使用；`csv`、`pathlib` 等标准库模块需要先导入。

标准库的组合使用

假设 `orders.csv` 中已有三笔清洗后的订单，金额分别为360.00、640.00和500.00元。

```
from pathlib import Path
import csv
from statistics import mean

file_path = Path("data") / "orders.csv"
amounts = []

with file_path.open(encoding="utf-8-sig", newline="") as file:
  reader = csv.DictReader(file)
    for row in reader:
      amounts.append(float(row["金额"]))

print(f"订单数: {len(amounts)}")
print(f"平均金额: {mean(amounts):.2f}元")
```

模块之间的分工： `pathlib` 负责路径，`csv` 负责读取字段，`statistics` 负责计算指标。每个模块只解决自己擅长的问题。

标准库与第三方库

类型	获取方式	示例
标准库	随Python一起提供，通常可以直接导入	`csv`、`pathlib`、`statistics`
第三方库	由其他开发者提供，通常需要先安装	`pandas`、`openpyxl`、`requests`

使用 `pandas` 读取订单文件并计算平均金额：

```
import pandas as pd

orders = pd.read_csv("data/orders.csv")
average = orders["金额"].mean()

print(f"平均金额: {average:.2f}元")
```

第三方库的作用

第三方库提供已经实现并经过广泛使用的功能。使用合适的库可以减少重复代码，但项目会增加新的外部依赖。安装名称和导入名称可能不同。例如，安装时使用scikit-learn，代码中通常使用import sklearn。

在当前Python环境中安装库

在Jupyter Notebook的代码单元中安装：

```
%pip install pandas
```

在本机终端中安装并查看安装位置：

```
python -m pip install pandas
python -m pip show pandas
```

在Notebook中查看当前使用的Python解释器：

```
import sys
print(sys.executable)
```

为什么推荐这些写法

Notebook中的%pip会面向当前内核运行安装命令。终端中的python -m pip可以明确使用与python命令对应的包管理工具。如果安装后仍然无法导入，先确认安装库和运行代码使用的是同一个Python环境。Notebook安装新库后，有时还需要重新启动内核。

每个项目使用独立环境

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows PowerShell

```
py -m venv .venv
.venv\Scripts\Activate.ps1
```

激活后检查当前环境

```
python -m pip --version
python -c "import sys; print(sys.executable)"
```

虚拟环境中保存什么

`.venv` 目录包含项目使用的Python解释器和已经安装的第三方库。激活后，终端中的 `python` 和 `pip` 会指向这个环境。

为什么每个项目单独创建

不同项目可以使用不同版本的第三方库，减少升级一个项目时影响其他项目的情况。

依赖记录让环境可以重建

在原项目环境中记录依赖

```
python -m pip install pandas
python -m pip freeze \
> requirements.txt
```

在新环境中恢复依赖

```
python -m pip install \
-r requirements.txt
python -m pip check
```

项目内容	是否需要保存和共享
项目代码、数据说明和 `requirements.txt`	需要保存。其他人可以据此重新创建项目环境。
`.venv` 目录	通常不提交或共享。其他人在自己的电脑上重新创建。

使用pip freeze的前提

在当前项目的独立虚拟环境中生成依赖文件。如果在包含许多无关软件包的公共环境中运行，文件也会记录这些无关依赖。

订单渠道汇总项目

项目内容	具体要求
输入数据	读取包含订单号、渠道、金额和折扣率的 `orders.csv` 。
业务处理	检查金额和折扣率，计算每笔订单的实付金额。
结果输出	跳过无效订单，按渠道汇总有效订单的实付金额。

```
order_project/
  data/
    orders.csv
  main.py
  order_utils.py
  report_utils.py
```

代码文件的功能要求和验证要求

`main.py` 组织流程， `order_utils.py` 完成订单计算， `report_utils.py` 完成渠道汇总。分别使用正常订单、无法转换的金额和超出范围的折扣率进行测试。确认有效订单进入汇总，无效订单显示原因并被跳过。

测试数据：orders.csv

```
订单号,渠道,金额,折扣率
O001,天猫,640.00,0.15
O002,京东,360.00,0.10
O003,天猫,500.00,0.20
O004,抖音,六百,0.10
O005,京东,800.00,1.20
O006,抖音,-100.00,0.05
```

有效订单

`O001`、`O002` 和 `O003` 能够正常计算实付金额，用于检查按渠道汇总的结果。

无效订单

`O004` 的金额无法转换，`O005` 的折扣率超出范围，`O006` 的金额为负数。程序应显示原因并跳过这些订单。

预期汇总结果

天猫渠道的实付金额合计为 `944.00` 元，京东渠道的实付金额合计为 `324.00` 元。

业务计算与渠道汇总模块

order_utils.py

```
def calculate_payable(order):
    amount = float(order["金额"])
    rate = float(order["折扣率"])
    if amount < 0:
        raise ValueError("订单金额不能为负数")
    if not 0 <= rate <= 1:
        raise ValueError("折扣率必须在0和1之间")
    return amount * (1 - rate)
```

report_utils.py

```
def summarize_by_channel(orders):
    totals = {}
    for order in orders:
        channel = order["渠道"]
        totals[channel] = (totals.get(channel, 0.0) + order["实付金额"])
    return totals
```

两个模块的分工：订单计算模块返回实付金额。主程序把结果保存到订单字典的"实付金额"字段，再交给报表模块汇总。

主程序连接各个模块

```
from pathlib import Path
import csv
from order_utils import calculate_payable
from report_utils import summarize_by_channel

path = Path("data") / "orders.csv"
valid_orders = []

with path.open(encoding="utf-8-sig", newline="") as file:
  for row in csv.DictReader(file):
    try:
      row["实付金额"] = calculate_payable(row)
    except (KeyError, ValueError) as error:
      order_id = row.get("订单号", "未知订单")
      print(f"跳过{order_id}: {error}")
      continue
    valid_orders.append(row)

summary = summarize_by_channel(valid_orders)
for channel, total in summary.items():
  print(f"{channel}: {total:.2f}元")
```

本讲知识结构

需要解决的问题	使用的方法	检查重点
单个文件过长	按职责拆分模块	每个文件是否负责一类清晰任务。
使用其他文件的功能	使用 `import` 导入	模块名称、所在目录和导入方式是否正确。
避免重复实现功能	查找Python标准库	标准库是否已经提供稳定的解决方案。
用外部开发者的代码	安装第三方库	安装位置是否与运行代码的Python环境一致。
让项目能够重新运行	使用虚拟环境和依赖文件	是否记录项目所需的第三方库及版本。

代码组织的基本顺序

先明确每部分代码的职责，再决定放入哪个模块。需要现成功能时，先查标准库，再评估第三方库。

组织后的主程序

主程序主要描述业务流程。具体的数据处理、业务计算和报表生成由相应模块中的函数完成。

课后实践：重构订单程序

实践任务

选择一个已经能够运行的订单处理程序，在不改变计算结果的前提下，将文件读取、业务计算和报表汇总代码拆分到不同模块中。

提交内容

- 包含 `main.py` 和至少两个自定义模块的项目文件夹；
- 一组正常数据和一组能够触发异常处理的数据；
- 原程序与重构程序的结果对比；
- 如果使用第三方库，提交 `requirements.txt` 。

使用AI智能体进行代码审查

可以让AI智能体检查模块职责、重复代码和导入关系。需要自己核对业务规则、测试结果和异常处理是否发生变化。

验收标准

主程序能够正常运行，各模块可以单独阅读，重构前后的有效订单汇总结果一致。

课后拓展：包的项目结构

项目结构

```
order_project/
  main.py
  sales_tools/
    __init__.py
    order_utils.py
    report_utils.py
```

sales_tools包

`sales_tools` 把订单计算和报表汇总模块组织在同一个目录中。

sales_tools/__init__.py

```
^^I^^Ifrom .order_utils import calculate_payable
^^I^^Ifrom .report_utils import summarize_by_channel
```

初始化文件的作用：`__init__.py` 把常用函数提供给包的使用者，形成统一的导入入口。

课后拓展：包的导入与使用

main.py

```
from sales_tools import (calculate_payable, summarize_by_channel)
```

如何理解这个结构

点号表示当前包中的模块。`__init__.py` 把常用函数提供给包的使用者，使主程序不必了解函数分别保存在哪个模块中。

何时需要使用包

模块较少时，直接使用多个 `.py` 文件已经足够。相关模块增多后，再使用包进行分组。

THE END